COMP229 Midterm - Equipment REST API

Student: Lodovic Rothney Feutse Nziko
Student ID: 301580784
GitHub repository: https://github.com/nidovic/COMP229-F26-MIDTERM

1. Overview

This project is a REST API built with Node.js and Express that manages an in-memory collection of equipment items (name, category, brand, year, quantity). It exposes six endpoints: get all items, filter by category, get by index, create, update and delete. Input is validated with the express-validator package, and invalid requests receive an HTTP 400 response with a list of errors. Unknown indexes receive HTTP 404.

2. Personalized equipment

My student ID ends in 4, so the two added items have release years between 2010 and 2019:

- Digital Audio Recorder - category audio, brand Zoom, year 2016, quantity 2.
- HD Video Camera - category video, brand Canon, year 2018, quantity 3.

Both items are defined in server.js and appear in the GET all response (screenshot 1).

3. Endpoints and test evidence

3.1 GET /api/equipment

Returns the full starting collection: 5 starter items plus my 2 personalized items. Status 200 OK.

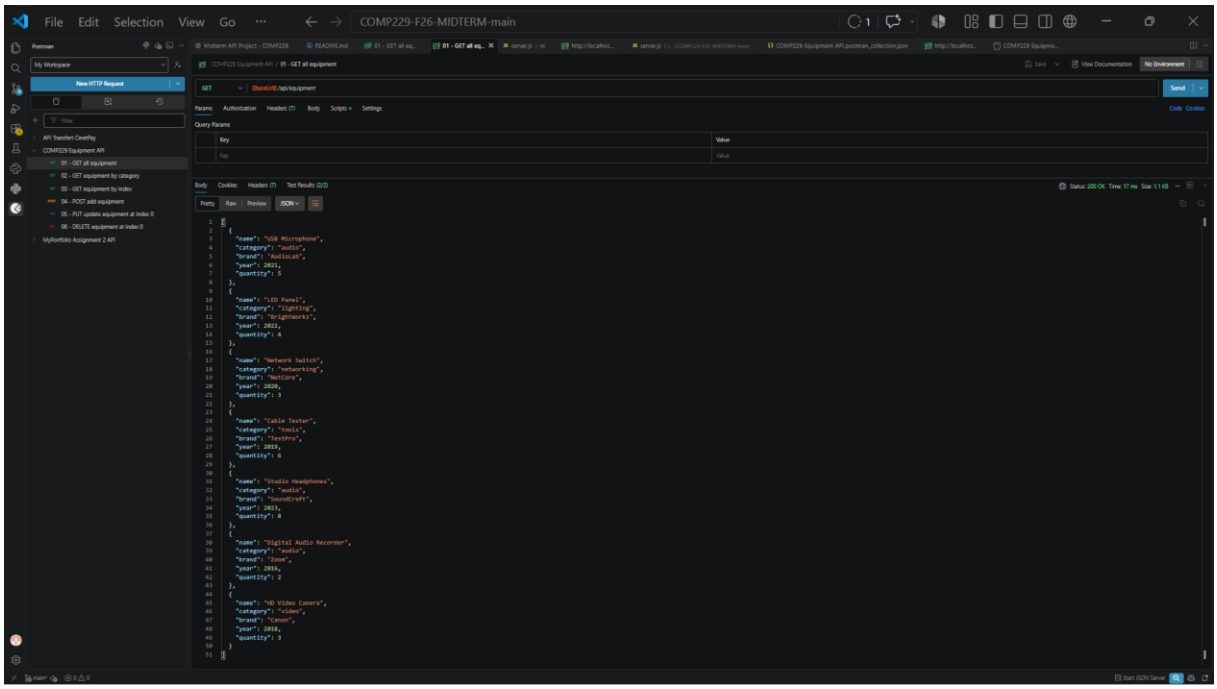


Figure 1: 301580784_FeutseNziko_01_GET_ALL.png

3.2 GET /api/equipment/filter?category=audio

Returns only the items whose category matches the query parameter. Status 200 OK.

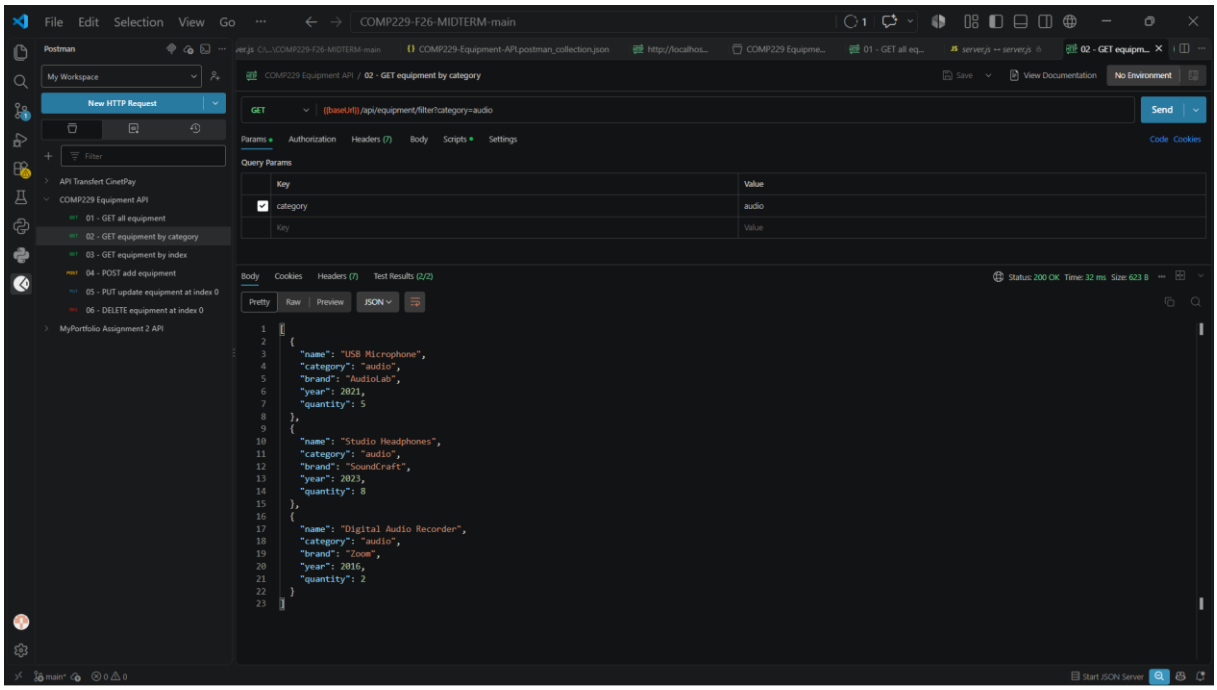


Figure 2: 301580784_FeutseNziko_02_GET_FILTER.png

3.3 GET /api/equipment/2

Returns the single item at array index 2. Status 200 OK.

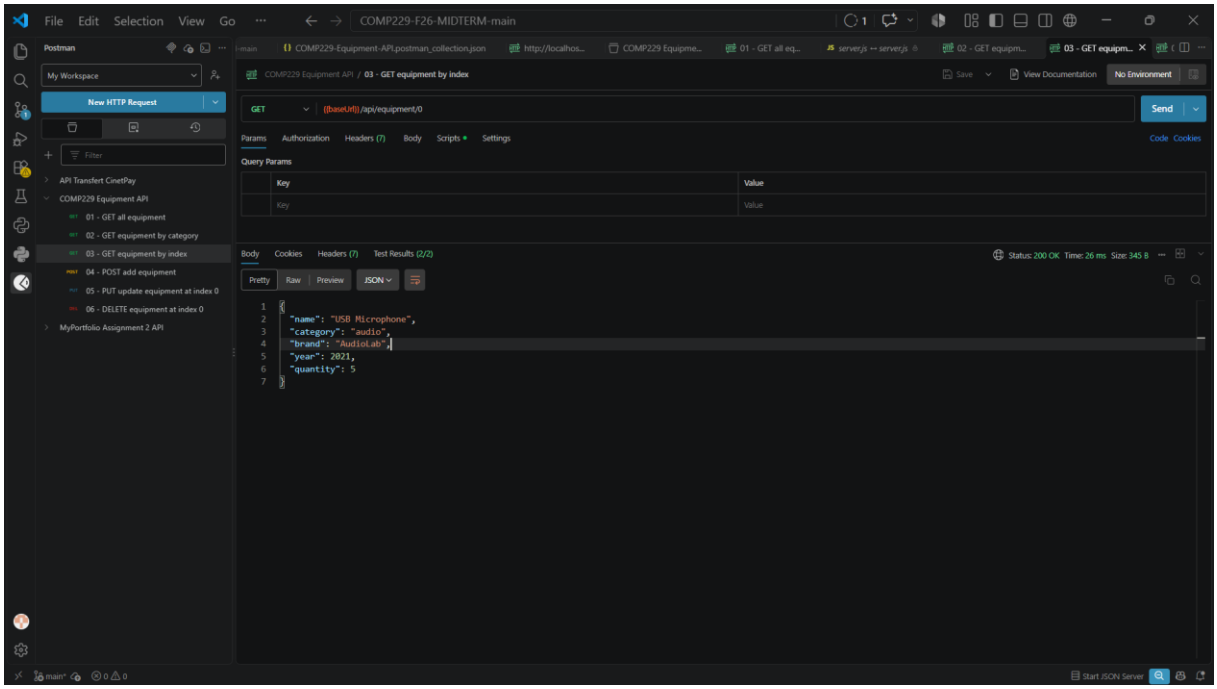


Figure 3: 301580784_FeutseNziko_03_GET_BY_ID.png

3.4 POST /api/equipment

Creates a new item from the JSON body and returns it. Status 201 Created.

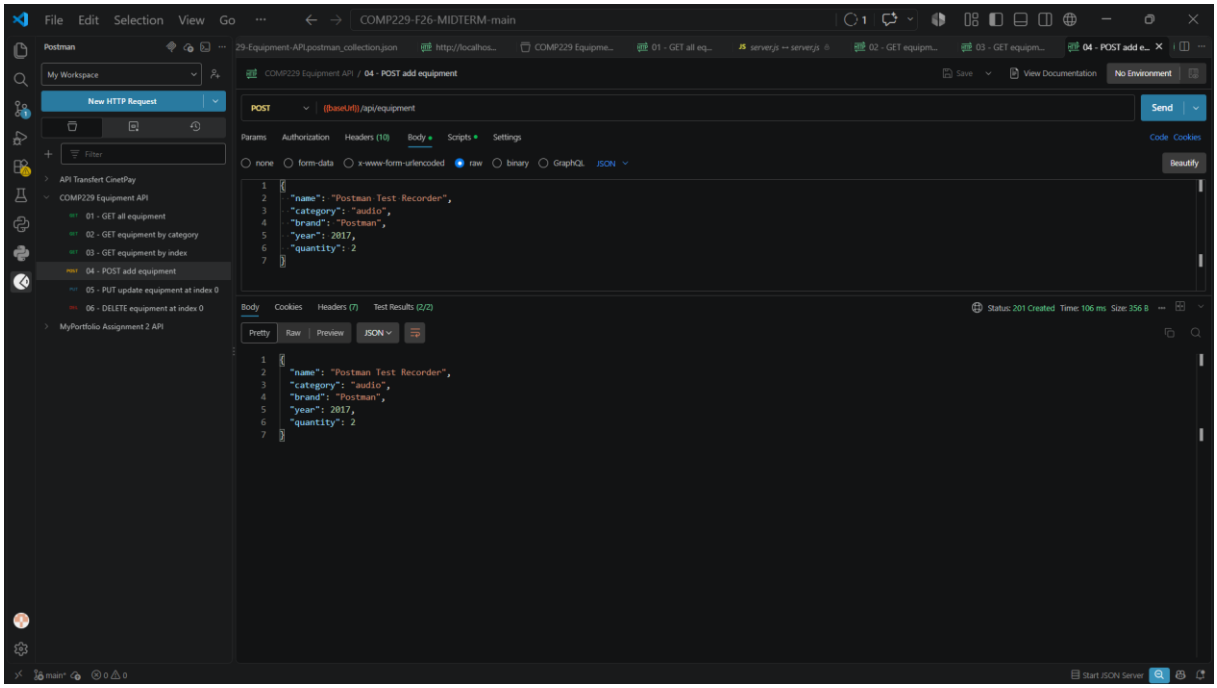


Figure 4: 301580784_FeutseNziko_04_POST.png

3.5 PUT /api/equipment/1

Updates the item at index 1 with the JSON body and returns the updated item. Status 200 OK.

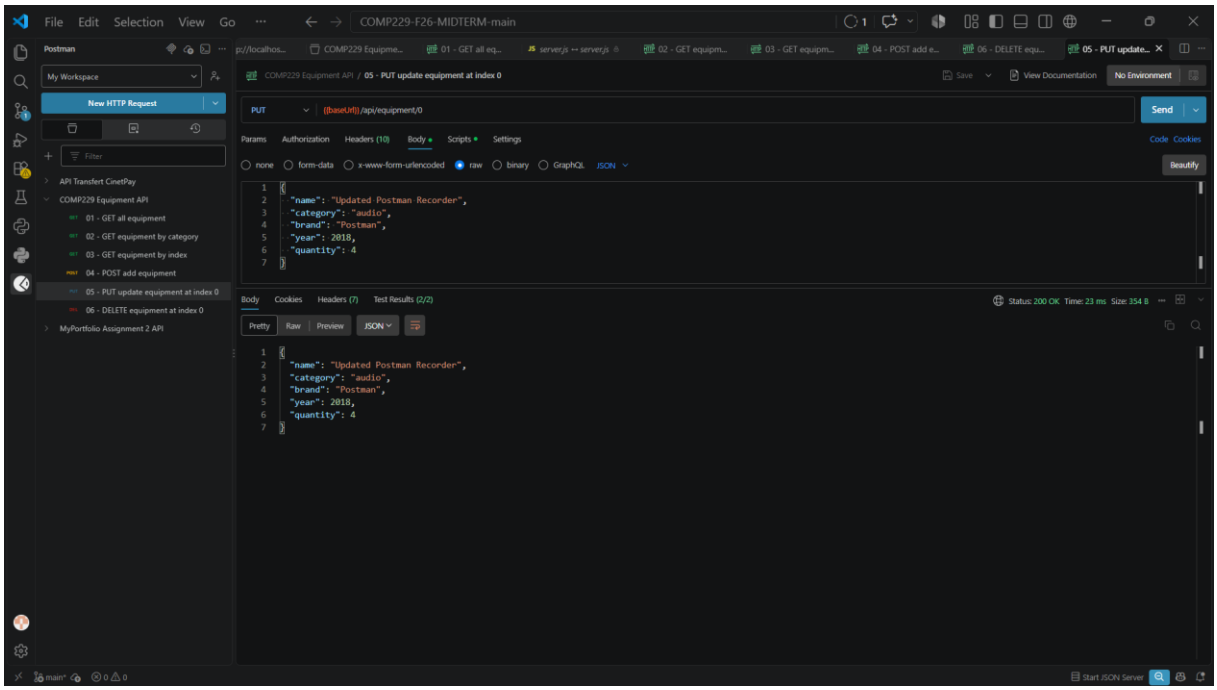


Figure 5: 301580784_FeutseNziko_05_PUT.png

3.6 DELETE /api/equipment/1

Removes the item at index 1 and returns the deleted item. Status 200 OK.

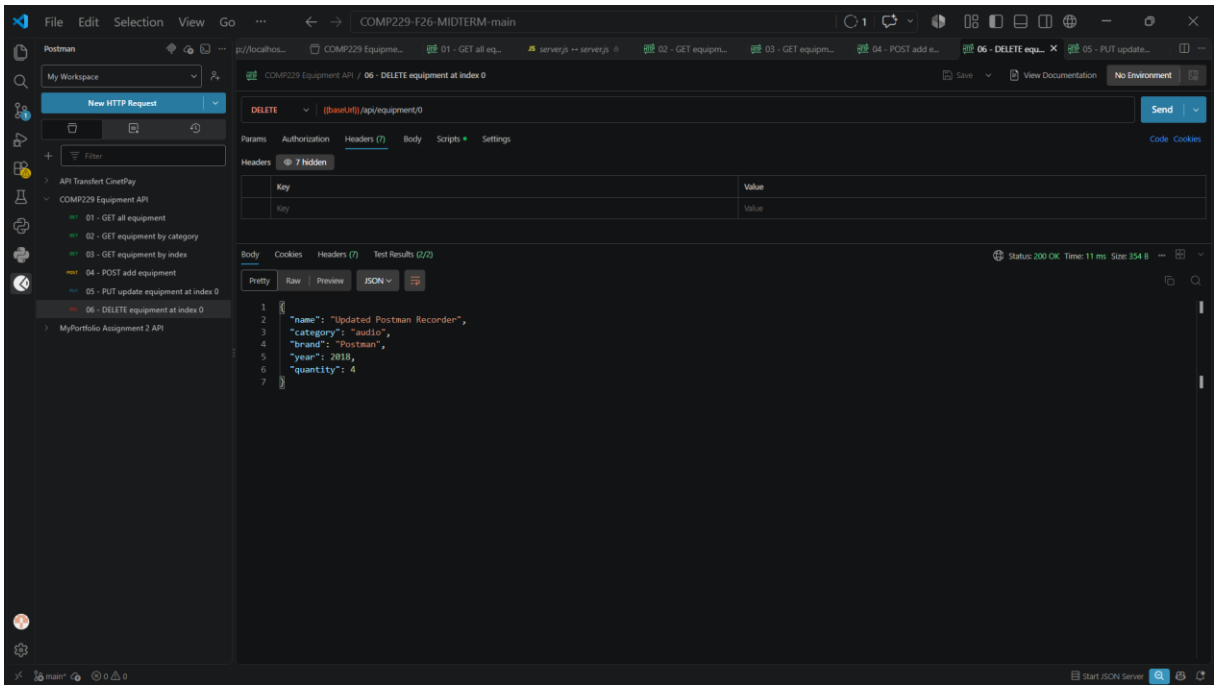


Figure 6: 301580784_FeutseNziko_06_DELETE.png

4. Reflection

I validated each endpoint with a Postman collection (one request per endpoint, with tests on the status code and response body) and with the automated suite run by npm test, which passes all 8 tests. The tests cover the seven-item starting collection, category filtering, lookup by index, and the create, update and delete operations.

Edge cases I considered: a non-numeric or out-of-range index (400 / 404), a missing or empty category in the filter, a missing field or a non-numeric quantity or year in POST and PUT (400 with validation errors), and the route order, since the filter route must be declared before the route with the :id parameter. Because the data lives in memory, I restarted the server before taking the GET all screenshot so it shows the clean initial collection.